FILE HANDLING IN PYTHON

File Handling

File Handling is the process of creating, reading, writing and managing files stored in a system.

File

A File is a collection of data stored on a storage medium.

```
file name.extension
```

`file_name` – name of the file
`extension` – type of the file

Program	File
contains instructions to perform a task	collection of data stored on a storage medium
loaded into RAM	stored on disk
Volatile – temporary memory, gone after execution	**Non-volatile** – persistent memory, survives restart
Faster access	Slower access

Types of Files

1. Text Files

Data is stored in a **human-readable** format. i.e. uses characters to store data. **Eg.** `.txt, .csv, .json` files.

2. Binary Files

Data is stored in **binary format.** Contains only 0s and 1s. Used to store multimedia data. **Eg.** `.jpg, .mp3, .mp4`

Path

Path is the location of a file in a system.

Absolute Path

Complete path from the root directory

```
C:\Users\Student\Documents\data.txt
```

Relative Path

Path relative to the current working directory

```
Documents\data.txt
```

Opening a File

`open()` function

```
f = open(file_name, mode)
```

`file_name` – name of the file
`mode` – the operation to be performed on the file

Mode	Operation
r	read a file
w	write into a file
a	append to a file
r+	read & write
w+	read & write
a+	read & append
rb	read a binary file
wb	write into a binary file
ab	append to a binary file

Closing a File

`close()` function

```
f.close()
```

`with open()` function

It automatically closes a file after use.

```
with open(file_name, mode) as f:
    #file operations
```

Reading a File

`r` – opens the file in read mode, throws an error if file does not exist

```
>>> f = open("data.txt", "r")
>>> f.close()
>>> f = open("sample.txt", "r")
Traceback (most recent call last):
  File "<python-input-9>", line 1, in <module>
    f = open("sample.txt", "r")
FileNotFoundError: [Errno 2] No such file or directory: 'sample.txt'
```

`read()` – reads the entire file into a single string. File pointer moves to the end of file.

```
>>> with open("data.txt", "r") as f:
...     content = f.read()
...     print(content)
...
Hello, My name is Hussain.
I'm a Bachelor in Computer Science.
I have a passion for teaching.
I'm good with subjects like Java, Python, C, C++, Front-end Technologies, Back-end Technologies, OOPS,
DSA, SQL.
```

`readline()` – read one line from the file. File pointer moves to the end of the line.

```
>>> with open("data.txt", "r") as f:
...     line1 = f.readline()
...     line2 = f.readline()
...     print(line1)
...     print(line2)
...
Hello, My name is Hussain.

I'm a Bachelor in Computer Science.
```

`readlines()` – reads the entire file as a list of strings, where each string represents one line from the file. File pointer moves to the end of file.

```
>>> with open("data.txt", "r") as f:
...     content = f.readlines()
...     print(content)
...
['Hello, My name is Hussain.\n', "I'm a Bachelor in Computer Science.\n", 'I have a passion for teachin
g. \n', "I'm good with subjects like Java, Python, C, C++, Front-end Technologies, Back-end Technologie
s, OOPS, DSA, SQL."]
```

Writing into a File

`w` – opens a file in write mode. Creates a new file if file does not exist, overwrites an existing file.

Creating a new file

```
>>> with open("sample.txt", "w") as f:
...     f.write("A new file is created")
...
21
>>>
```

sample - Notepad

File Edit Format View Help

A new file is created

Overwriting existing file

```
>>> with open("sample.txt", 'w') as f:
...     f.write("Data is over written")
...
20
>>>
```

sample - Notepad

File Edit Format View Help

Data is over written

`write()` – writes a string into a file
`writelines()` – writes a list of strings into a file
Both these functions don't add a new line at the end automatically.

```
>>> with open("sample.txt", "w") as f:
...     f.write("The write function is used to write into a file\n")
...     lines = ["Hello ", "World!\n", "My name is Hussain\n", "This is my Python programming notes"]
...     f.writelines(lines)
...
48
>>>
```

```
The write function is used to write into a file
Hello World!
My name is Hussain
This is my Python programming notes
```

Appending a File

a – opens the file in append mode. Creates a new file if file does not exists, adds data at the end of the file without overwriting if file exists already.

```
>>> with open('sample.txt', 'a') as f:
...     f.write('\nThe file is opened in append mode and does not erase the existing data')
...
71
>>>
```

```
The write function is used to write into a file
Hello World!
My name is Hussain
This is my Python programming notes
The file is opened in append mode and does not erase the existing data
```

Update modes

r+ - opens the file in read and write mode, throws an error if file does not exist. It does **not overwrite the entire file**. Instead, it starts writing from the **beginning of the file** and **replaces only the characters that are written**, leaving the remaining content unchanged.

```
>>> 'Before overwriting'
'Before overwriting'
>>> with open('sample.txt','r+') as f:
...     print(f.read())
...
PythonWorld!
>>> 'overwriting with r+ mode'
'overwriting with r+ mode'
>>> with open('sample.txt', 'r+') as f:
...     f.write('Hello ')
...
6
>>> 'After overwriting'
'After overwriting'
>>> with open('sample.txt','r') as f:
...     print(f.read())
...
Hello World!
```

w+ - opens the file in read and write mode. It creates a new file if it does not exist, and overwrites the existing data if file already exists.

a+ - opens the file in read and append mode. It creates a new file if it does not exist, and adds data to the end of the file without erasing the existing content.

File Pointer

A File pointer is a position indicator that specifies the location in which the next file operation is going to take place.

`f.tell()` – gives the current position of the cursor in the file.

`f.seek(index)` – moves the cursor to the specified index.

Common File Handling errors

`FileNotFoundError` – occurs when a file that does not exist is opened in read mode

`PermissionError` – occurs when the program does not permission to access a file

`ValueError` – occurs when I/O operations are performed on a closed file

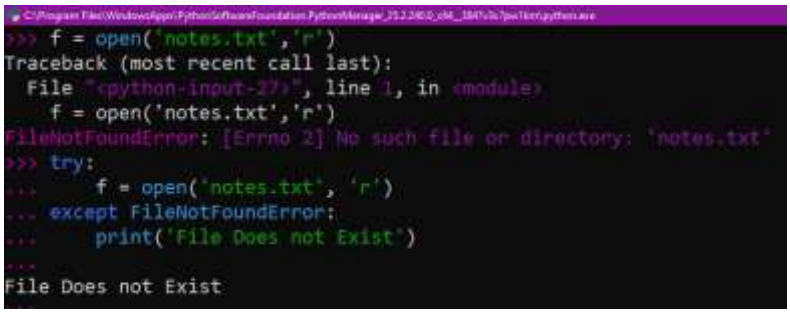
`IsADirectoryError` – occurs when a file operation is performed on a directory

`NotADirectoryError` – occurs when a directory operation is performed on a file

Using `try-except` for safe file handling

`try` – contains risky code that may throw an error.

`except` – contains error handling code that handles errors and prevent program crash.

Syntax

```
try:
      #Risky code
except ExceptionType:
      #error handling code
```